

去哪儿网高级前端工程师 1-3面

第一轮：技术面 (时长：约60分钟)

1. 我看你简历里写到对项目做了很多性能优化，那你们当时是怎么衡量优化的效果的？主要关注哪些性能指标？比如 Core Web Vitals 里的 LCP 和 INP，在你的项目中分别代表什么？
2. 针对你刚才提到的 LCP 指标，如果一个页面的 LCP 过高，你会从哪些方面入手去分析和解决？请结合我们机票搜索结果页这种场景具体谈谈。
3. 当列表数据量非常大时，一次性渲染所有 DOM 节点肯定会造成页面卡顿。你了解虚拟列表 (Virtual List) 的实现原理吗？大概描述一下它的核心思路。
4. `let`，`const` 和 `var` 在作用域、变量提升和暂时性死区方面有什么区别？在 `for` 循环中使用 `let` 和 `var` 定义循环变量，会产生什么不同的结果？
5. `null` 和 `undefined` 有什么区别？在什么场景下你会主动使用 `null`？
6. 好的，我们来写一道代码题吧。在线旅游网站经常需要处理各种复杂的数据，这道题就跟它相关。
 - 题目：“假设有一个航线列表数组 `flightList`，每个元素是一个对象，包含 `price` (价格)，`duration` (飞行总时长，单位分钟)，`stops` (经停次数)。请实现一个排序函数 `sortFlights(list, sortBy, order)`，可以根据指定的字段（如 `'price'`）和顺序（`'asc'` 或 `'desc'`）对列表进行排序。请重点考虑当两个元素的排序字段值相同时，如何保持其原始相对顺序（即实现一个稳定的排序）。”
7. （写完后）“可以讲解一下你的实现思路吗？如果你要支持多个排序条件，比如先按价格升序，价格相同的再按飞行时长升序，你会如何扩展你的函数？”

第二轮：技术面 (时长：约60分钟)

1. 请深入介绍一下你简历上那个最能体现你技术能力的复杂项目。它的业务背景是什么？你在其中承担了什么样的角色，解决了哪些核心的技术难题？
2. 在那个项目中，你们的团队是如何进行前端状态管理的？对于像我们搜索条件筛选区那样，存在大量查询参数并且参数间可能存在联动关系的场景，你是如何设计其状态的读取、派发和持久化的？
3. 你在项目中是否处理过兼容性问题？可以举一个你印象最深刻的浏览器兼容性 case 吗？你是如何定位和解决的？
4. 追问几个细节。针对这个列表页，数据量非常大，你设计的数据获取策略是怎样的？是一次性请求所有数据，还是分页加载，或者是其他方式？各有什么优缺点？
5. 用户在左侧的筛选器中（比如选择航司、起飞时段、价格区间），列表需要实时更新。这个筛选功能的逻辑你打算放在前端还是后端实现？为什么？如果放在前端，如何保证筛选的性能？

6. 当用户向下滚动页面时，如何实现“无限滚动”加载更多数据？在实现这个功能时，有哪些需要注意的性能和体验上的细节？比如如何防止重复发送请求？
7. 最后一个问题，对于这个页面的可维护性，你会如何设计你的组件结构？比如单个航线卡片、筛选器、排序栏等，你会如何把它们抽象成可复用、低耦合的组件？

第三轮：总监面 (时长：约45-50分钟)

1. 你如何看待 Server-Side Rendering (SSR) 和传统的 Client-Side Rendering (CSR)？对于像去哪儿网这样的OTA网站，你认为在哪些页面采用SSR会带来比较大的业务价值？为什么？
2. 我们的移动端流量占比非常高。对于移动端的前端技术方案，你对H5、小程序、React Native/Flutter这类技术方案的看法是怎样的？如果让你来为我们的酒店预订流程选择一个移动端方案，你会怎么决策？
3. 在一个像我们这样已经发展了十多年的公司里，不可避免地会存在一些历史技术债。你如何看待技术债？作为一名高级工程师，你觉得应该如何平衡新业务的快速迭代和偿还技术债之间的关系？
4. 你认为一个优秀的高级前端工程师，除了技术能力之外，最重要的特质是什么？在过往的团队中，你是如何发挥自己的影响力，帮助团队和身边的同事一起成长的？
5. 你近期的职业规划是什么？你希望未来成为一个在特定领域深耕的技术专家，还是一个懂技术、懂业务的团队管理者？

一面参考答案：

1、我看你简历里写到对项目做了很多性能优化，那你们当时是怎么衡量优化的效果的？主要关注哪些性能指标？比如 Core Web Vitals 里的 LCP 和 INP，在你的项目中分别代表什么？

我们衡量优化的效果，会结合“**实验室数据**”和“**真实用户数据（RUM）**”两个维度，确保优化是全面且有效的。

- **实验室数据（Lab Data）**：在开发和测试阶段，我们主要使用 **Chrome DevTools** 里的 **Lighthouse** 和 **Performance** 面板。这能提供一个稳定、可复现的性能基线，方便我们在优化前后进行对比，快速验证某个改动是否有效。
- **真实用户数据（Real User Monitoring, RUM）**：上线后，我们更关注真实用户的数据，因为这反映了在各种网络、设备和使用场景下的实际性能。我们使用像 Sentry、Datadog 这样的监控平台来收集和分析这些数据。这能帮助我们发现实验室环境中难以复现的问题。

我们主要关注以下几类性能指标：

1. **加载性能**：这直接关系到用户的第一印象。核心就是 **Core Web Vitals (CWV)**。
 - **LCP (Largest Contentful Paint)**：最大内容绘制。

- **INP (Interaction to Next Paint)**: 下次绘制交互。这是谷歌新推出的、用于替代FID的指标，衡量的是整体的交互响应性。
- **CLS (Cumulative Layout Shift)**: 累积布局偏移。

2. 运行时性能:

- **FPS (Frames Per Second)**: 动画或滚动时的帧率，低于60fps就会感觉卡顿。
- **主线程阻塞时间 (Main-thread Blocking Time)**: 长时间运行的JS任务会阻塞主线程，导致页面无响应。

3. 资源指标:

- **Bundle Size**: JS、CSS等资源包的体积，直接影响加载速度。
- **请求数量**: 页面加载所需的网络请求次数。

LCP 和 INP 在我项目中的具体体现

在我之前负责的项目中:

- **LCP (最大内容绘制)** 指标主要衡量的是**商品主图**的加载速度。因为在商品详情页，用户视口内最大、最核心的内容通常就是那张高清的商品主图。LCP时间过长，用户就会感觉页面“出不来”，直接影响转化率。
- **INP (下次绘制交互)** 在这个页面上则代表了多种交互的响应速度。例如:
 - 用户点击**“切换商品规格”（如颜色、尺码）**按钮后，页面能否立刻响应，更新价格、库存和主图。
 - 用户点击**“添加到购物车”**按钮后，按钮是否能立即变为“已添加”或显示一个加载动画，而不是延迟几百毫秒才有反应。
 - 用户点击**图片缩略图**切换查看时，大图是否能流畅地更新。
 - INP衡量的是这些交互的端到端延迟，如果延迟高，用户会感觉页面“卡”或“不跟手”，严重影响购物体验。

2、针对你刚才提到的 LCP 指标，如果一个页面的 LCP 过高，你会从哪些方面入手去分析和解决？请结合我们机票搜索结果页这种场景具体谈谈。

如果一个机票搜索结果页的LCP过高，我会遵循一个清晰的分析路径来定位和解决问题。

第一步：确定LCP元素是什么

首先，我会使用Chrome DevTools的Performance面板录制一次页面加载，在时间轴上找到LCP标记，确认LCP元素到底是什么。在机票搜索结果页，这个元素极大概率是机票列表本身，或者是顶部的某个大型的宣传横幅(Banner)。

第二步：分析LCP的四个关键阶段

LCP的延迟主要来自四个阶段，我会逐一排查：

1. TTFB (Time to First Byte) 过长 - 服务器响应慢

- **问题：**从浏览器发起请求到收到服务器返回的第一个字节的时间太长。这说明瓶颈在后端。
- **分析：**查看网络请求瀑布图，看主文档的TTFB时间。
- **机票页场景解决方案：**
 - **后端优化：**检查机票查询接口的性能，是否数据库查询过慢或第三方航司接口响应慢。
 - **缓存：**对于热门航线（如“东京-大阪”），可以在后端或使用Redis等缓存查询结果。
 - **使用CDN：**将静态资源和API网关部署在CDN上，让用户从最近的节点访问。

2. 资源加载延迟和阻塞 - 关键资源路径长

- **问题：**浏览器在渲染LCP元素前，被一些不必要的CSS或JS文件阻塞了。
- **分析：**检查 `<head>` 标签里的 `<script>` 和 `<link>` 标签。
- **机票页场景解决方案：**
 - **CSS优化：**将渲染列表所必需的“关键CSS”内联到HTML中，或使用HTTP/2 Server Push。非关键的CSS（如弹窗、页脚的样式）则进行异步加载。
 - **JS优化：**为所有非必须的JS脚本添加 `defer` 或 `async` 属性，防止它们阻塞HTML的解析。特别是第三方统计、广告脚本。

3. LCP元素自身加载缓慢

- **问题：**LCP元素本身（如列表的背景大图、或获取列表数据的API请求）下载很慢。
- **分析：**在网络瀑布图中找到LCP元素或其依赖的数据请求，看它的加载耗时。
- **机票页场景解决方案：**
 - **如果是Banner图：**对图片进行压缩，使用WebP等现代格式，并为不同分辨率的设备提供不同尺寸的图片。使用 `<img>` 的 `fetchpriority="high"` 属性提升其加载优先级。
 - **如果是机票列表数据：**这个API请求至关重要。可以使用 `preconnect` 或 `dns-prefetch` 来预先建立与API服务器的连接，减少TCP和DNS握手时间。

4. 客户端渲染耗时长

- **问题：**页面采用纯客户端渲染（CSR），浏览器下载完一个空的HTML和巨大的JS包后，需要执行JS、再发起API请求获取机票数据、最后才把列表渲染出来。这个过程非常漫长。
- **分析：**查看Performance面板，看JS的执行（Scripting）时间是否过长。
- **机票页场景解决方案：**

- **采用服务端渲染 (SSR)**：这是解决机票搜索页这类动态内容LCP问题的**最佳方案**。服务器直接返回包含了前10-20条机票信息的完整HTML，浏览器一收到就能立刻渲染出LCP元素（机票列表）。后续的交互再由客户端JS接管。
 - **代码分割 (Code Splitting)**：将JS包按需分割。机票结果页不需要预加载订单填写页或用户中心的JS代码。
-

3、当列表数据量非常大时，一次性渲染所有 DOM 节点肯定会造成页面卡顿。你了解虚拟列表（Virtual List）的实现原理吗？大概描述一下它的核心思路。

当然了解。虚拟列表是解决海量数据渲染性能问题的关键技术，其核心思路非常巧妙：**只渲染用户当前视口（Viewport）内可见的列表项。**

它的实现原理可以概括为以下几点：

1. 计算总高度，撑开容器：

- 首先，我们需要一个外层的容器（`outer-container`）。它的高度不是由内容撑开的，而是我们通过JS计算出来的“**总列表的理论高度**”（`总项目数 × 每个项目的预估高度`）。
- 这样做唯一的目的是，为了在页面上**撑出一个“真实”的滚动条**，让用户可以正常地滚动。

2. 创建渲染窗口：

- 在这个外层容器内部，我们创建一个用于**实际渲染DOM节点**的内层容器（`inner-container` 或 “渲染窗口”）。这个容器通常使用绝对定位（`position: absolute`）。

3. 监听滚动事件：

- 我们监听外层容器的 `scroll` 事件。当用户滚动时，我们可以获取到当前的滚动偏移量 `scrollTop`。

4. 动态计算和渲染：

- 在滚动事件的回调函数中，执行核心的计算逻辑：
 - **计算可见区域**：根据 `scrollTop` 和视口的高度，计算出当前应该渲染的列表项的起始索引（`startIndex`）和结束索引（`endIndex`）。为了有更好的滚动体验，通常还会在可见区域的上下增加一些“缓冲区”（Buffer）。
 - **截取数据**：从完整的海量数据源数组中，只截取 `startIndex` 到 `endIndex` 这一小部分数据。
 - **更新渲染**：将截取出来的这一小部分数据，传递给内层容器进行渲染。

- **移动窗口**：同时，通过CSS的 `transform: translateY()` 属性，将内层容器（渲染窗口）整体向下移动 `startIndex × 每个项目的高度` 的距离。这使得这一小段渲染出的DOM，能够精确地出现在滚动条所在的正确视觉位置上。

总结起来：用户看到的仿佛是一个完整的长列表，但实际上DOM中永远只存在几十个节点。用户滚动时，我们只是在动态地替换这几十个节点的数据，并快速地移动这个“渲染窗口”的位置，从而用极小的渲染开销，模拟出了无限滚动的效果。

5. `let`, `const` 和 `var` 在作用域、变量提升和暂时性死区方面有什么区别？在 `for` 循环中使用 `let` 和 `var` 定义循环变量，会产生什么不同的结果？

这三者是JS中声明变量的关键方式，它们的区别是ES6+面试中的核心知识点。

特性	var	let	const
作用域 (Scope)	函数作用域 (Function Scope)	块级作用域 (Block Scope {})	块级作用域 (Block Scope {})
变量提升 (Hoisting)	会提升，并初始化为 <code>undefined</code>	会提升，但不初始化	会提升，但不初始化
暂时性死区 (TDZ)	无	有。在声明前访问会报 <code>ReferenceError</code>	有。在声明前访问会报 <code>ReferenceError</code>
重复声明	允许在同一作用域内重复声明	不允许在同一作用域内重复声明	不允许在同一作用域内重复声明
修改值	可以修改	可以修改	不可修改（对于基本类型），但对象/数组的内部属性可修改

for 循环中的不同结果

这是一个非常经典的例子，能很好地体现 `var` 和 `let` 在作用域上的核心区别。

使用 `var`：

JavaScript

代码块


```
1 for (var i = 0; i < 3; i++) {
2   setTimeout(() => {
3     console.log(i);
4   }, 100);
5 }
6 // 输出结果:// 3// 3// 3
```

- **原因：**`var` 是函数作用域。在整个循环中，**只存在一个变量 `i`**。`setTimeout` 是异步的，当它里面的回调函数准备执行时，`for` 循环早已执行完毕。此时，变量 `i` 的最终值是 `3`。所以，所有的回调函数在执行 `console.log(i)` 时，引用的都是这个值为 `3` 的同一个变量 `i`。

使用 `let`：

JavaScript

代码块

```
1 for (let i = 0; i < 3; i++) {
2   setTimeout(() => {
3     console.log(i);
4   }, 100);
5 }
6 // 输出结果:// 0// 1// 2
```

- **原因：**`let` 是块级作用域。在 `for` 循环中使用 `let` 时，JavaScript引擎有一个特殊的行为：它会在**每一次循环迭代时，都创建一个新的、独立的块级作用域，并为该次迭代创建一个新的变量 `i`**。所以，`setTimeout` 的回调函数在每次迭代中“捕获”到的，都是那个特定迭代作用域下的、值为 `0`、`1`、`2` 的不同的变量 `i`。这通常是我们所期望的结果。

5、null 和 undefined 有什么区别？在什么场景下你会主动使用 null？

`null` 和 `undefined` 都表示“没有值”，但它们的含义和用法有本质区别。

- **`undefined`**：代表一个变量**已被声明，但尚未被赋值**。它是一种“系统级”的、**无意识**的空值。
 - 变量声明了但没赋值：`let a; console.log(a); // undefined`
 - 对象没有某个属性：`const obj = {a: 1}; console.log(obj.b); // undefined`
 - 函数没有返回值：`function foo() {} console.log(foo()); // undefined`
- **`null`**：代表一个变量被**人为地、显式地**赋予了“空值”。它是一种“程序级”的、**有意识**的空值，表示“这里曾经可能有值，但现在没有”或“这里应该有一个对象，但目前没有”。

主动使用 `null` 的场景：

1. **作为对象的初始值：** 当一个变量未来将要指向一个对象，但在初始化时还无法获取那个对象时，我会主动将其设为 `null`。这在React中非常常见。

2. JavaScript

代码块

```
1  const elementRef = useRef(null); // 初始化一个DOM引用// ... 在useEffect中，
   elementRef.current会被设为DOM对象
```

1. **作为函数参数的默认值或占位符：** 表明这个参数是可选的，或者是有意不传递的。
2. **表示对象已清空：** 当一个对象不再需要，为了让垃圾回收机制（Garbage Collector）能尽快回收其内存，可以将其引用显式地设置为 `null`。 `let myObj = { ... }; myObj = null;`
3. **与API的契约：** 在与后端API交互时，如果某个字段（比如用户的“个人简介”）是可选的，用户没有填写，后端返回 `{ "bio": null }` 比不返回这个字段（导致前端拿到 `undefined`）要更明确。它清晰地告诉前端：“我们有‘bio’这个字段，但它的值就是空”，而不是“可能后端出错了，忘了给我这个字段”。

6 & 7、代码题：实现一个稳定的排序函数，并讨论如何扩展

好的，没问题。

实现 `sortFlights`

代码块

```
1  /**
2   * 对航线列表进行稳定的排序
3   * @param {Array<Object>} list - 航线列表
4   * @param {string} sortBy - 排序依据的字段名，如 'price', 'duration', 'stops'
5   * @param {'asc' | 'desc'} order - 排序顺序，'asc' 为升序，'desc' 为降序
6   * @returns {Array<Object>} - 排序后的新数组
7   */function sortFlights(list, sortBy, order) {
8   // 1. 为了不修改原数组，我们先创建一个副本const listCopy = [...list];
9   }
```



```

10 // 2. 为了实现稳定排序，我们将数组元素和它的原始索引绑定在一起。// 这创建了一个中间
    数据结构，方便我们在排序值相同时进行比较。const mapped = listCopy.map((item, index)
    => ({ item, index }));
11
12 // 3. 核心排序逻辑
13 mapped.sort((a, b) => {
14     const valA = a.item[sortBy];
15     const valB = b.item[sortBy];
16
17     // 确定排序方向的乘数const orderMultiplier = order === 'asc' ? 1 : -1;
18
19     // 比较主排序字段if (valA < valB) {
20         return -1 * orderMultiplier;
21     }
22     if (valA > valB) {
23         return 1 * orderMultiplier;
24     }
25
26     // 4. 实现稳定排序的关键：当主排序字段值相等时，// 我们比较它们的原始索引。索引小的
    排在前面，保持了原始的相对顺序。return a.index - b.index;
27 });
28
29 // 5. 将排序后的中间结构，映射回原始的元素数组return mapped.map(el => el.item);
30 }
31
32 // --- 示例 ---const flightList = [
33     { id: 1, price: 500, duration: 120, stops: 1 },
34     { id: 2, price: 450, duration: 180, stops: 2 },
35     { id: 3, price: 500, duration: 100, stops: 0 }, // price与id:1相同
36     { id: 4, price: 400, duration: 150, stops: 1 },
37 ];
38
39 console.log("按价格升序（稳定）");
40 // 预期顺序：4, 2, 1, 3 (因为1在3前面，所以排序后也应该在前面)console.log(sortFlights(flightList, 'price', 'asc').map(f => f.id));
41
42 console.log("按价格降序（稳定）");
43 // 预期顺序：1, 3, 2, 4console.log(sortFlights(flightList, 'price',
    'desc').map(f => f.id));

```

实现思路讲解

我的实现思路核心在于**保证排序的稳定性**。虽然现代浏览器的 `Array.prototype.sort()` 大多已经是稳定排序，但在面试或编写需要跨环境运行的库时，不能依赖这个实现细节。

1. **创建副本**：首先，通过扩展运算符 `...` 创建原数组的副本，这是一个好习惯，避免了对原始数据的意外修改（副作用）。
2. **绑定原始索引**：我没有直接对副本排序，而是通过 `.map()` 方法，创建了一个新的、包含 `{ item, index }` 结构的数组。这一步是实现稳定排序的**关键准备**。它将每个元素的值和它在原始数组中的位置绑定在了一起。
3. **排序比较**：
 - 在 `.sort()` 的回调函数中，我首先比较主要的排序字段 `a.item[sortBy]` 和 `b.item[sortBy]`。
 - 为了方便处理升序(`asc`)和降序(`desc`)，我用了一个 `orderMultiplier` 变量。如果是升序，它是 `1`；如果是降序，它是 `-1`。这样，所有的比较结果乘以这个乘数，就能轻松反转排序方向，代码更简洁。
4. **稳定性保证**：当两个元素的主排序字段值相等时（`valA === valB`），我的比较函数并不会立刻返回 `0`。而是会进入**第二轮比较**：比较它们的原始索引 `a.index - b.index`。因为索引值是递增且唯一的，所以这个比较总能分出先后。索引小（即原始位置靠前）的元素，其比较结果也会更小，因此会被排在前面。这就**保证了原始相对顺序不被打乱**。
5. **还原结果**：最后，通过 `.map()` 将排序好的中间结构数组，再转换回只包含原始元素的数组，得到最终结果。

如何扩展以支持多个排序条件

要支持多个排序条件，比如先按价格升序，价格相同的再按飞行时长升序，我会扩展 `sortFlights` 函数，让 `sortBy` 和 `order` 可以接收一个**排序规则数组**。

JavaScript

代码块

```
1  /**
2   * 扩展后的排序函数，支持多条件排序
3   * @param {Array<Object>} list
4   * @param {Array<{field: string, order: 'asc' | 'desc'}>} criteria - 排序规则数组
5   * @returns {Array<Object>}
6   */function extendedSortFlights(list, criteria) {
7   // 绑定索引的步骤和之前一样const mapped = [...list].map((item, index) => ({
8     item, index }));
```

```

9 mapped.sort((a, b) => {
10     // 遍历排序规则数组for (let i = 0; i < criteria.length; i++) {
11         const { field, order } = criteria[i];
12         const valA = a.item[field];
13         const valB = b.item[field];
14         const orderMultiplier = order === 'asc' ? 1 : -1;
15
16         // 如果当前条件能分出先后，则立刻返回结果if (valA < valB) {
17             return -1 * orderMultiplier;
18         }
19         if (valA > valB) {
20             return 1 * orderMultiplier;
21         }
22         // 如果当前条件的值相等，则不返回，继续下一个条件的比较
23     }
24
25     // 如果遍历完所有条件都相等，则使用原始索引来保证稳定性return a.index - b.index;
26 });
27
28 return mapped.map(el => el.item);
29 }
30
31
32 // --- 扩展后调用示例 ---const multiSortCriteria = [
33     { field: 'price', order: 'asc' }, // 主要条件：价格升序
34     { field: 'duration', order: 'asc' }, // 次要条件：时长升序
35 ];
36
37 console.log("\n按价格升序，再按时长升序:");
38 // 预期：4, 2, 3, 1 (因为3的时长100 < 1的时长
120)console.log(extendedSortFlights(flightList, multiSortCriteria).map(f =>
f.id));

```

扩展思路：

核心改动在 `.sort()` 的回调函数内部。我不再只比较一个字段，而是用一个 `for` 循环来遍历传入的 `criteria` 排序规则数组。

- 在循环中，我按顺序依次取出每个排序条件（先是价格，再是时长）。
- 如果使用当前条件能分出先后（即 `valA` 和 `valB` 不相等），那么比较函数就立即返回结果，后续的排序条件将不再被考虑。
- 如果使用当前条件无法分出先后（即 `valA` 和 `valB` 相等），循环会继续，进入下一个排序条件来进行“决胜局”比较。
- 如果所有排序条件都比较完了，两个元素依然相等，最后我仍然会用它们的原始索引来做最终的比较，以确保整个多条件排序同样是稳定的。

这种结构非常灵活，可以轻松扩展到任意数量的排序条件。

二面参考答案：

1、请深入介绍一下你简历上那个最能体现你技术能力的复杂项目。它的业务背景是什么？你在其中承担了什么样的角色，解决了哪些核心的技术难题？

我的角色与解决的核心难题：

我作为该项目的前端技术负责人，除了负责核心业务模块的开发，主要解决了以下几个技术难题：

- 1. 前端实时策略引擎的设计与实现：**这是项目最复杂的部分。后端的差旅政策是高度动态、可配置的，前端需要接收一个复杂的规则集（可能是JSON格式的AST），并在海量的搜索结果（上百条机票/酒店）上**实时、高性能**地执行这些规则，然后更新UI。我设计了一套前端规则解释器，通过将规则解析成函数，并利用Web Worker进行计算，避免了复杂计算对主线程的阻塞，保证了页面交互的流畅性。
- 2. 高性能、高可用的搜索列表页架构：**机票酒店搜索页是核心，要求高性能和良好的用户体验。我主导设计了支持“无限滚动+虚拟列表”的数据获取和渲染方案，实现了在保证首屏快速加载的前提下，流畅展示上千条数据。这部分我们后面可以详细聊。
- 3. 复杂状态管理方案的设计：**搜索页的筛选条件非常复杂（涉及价格、时间、航司、中转、政策等多达几十个参数），且参数间存在联动关系。我设计了一套基于**URL作为唯一可信源**的状态管理模式，极大地简化了状态的同步、持久化和跨组件通信问题。

2、在那个项目中，你们的团队是如何进行前端状态管理的？对于像我们搜索条件筛选区那样，存在大量查询参数并且参数间可能存在联动关系的场景，你是如何设计其状态的读取、派发和持久化的？

对于这个场景，我们放弃了将所有筛选条件都塞进Redux或组件 `useState` 里的传统做法，因为这会导致状态同步和持久化变得异常复杂。我设计并推行了****以URL查询参数（URL Query Params）作为唯一可信数据源（Single Source of Truth）****的方案。

设计思路如下：

- **状态的读取 (Reading)：**
 - 组件在初始化时，唯一的任务就是读取当前页面的URL。我们使用 `react-router` 提供的 `useSearchParams` 这个Hook来解析URL中的查询参数，并将其作为筛选器组件的初始状态。
 - 例如，如果URL是 `/search/flights?price=0,5000&airline=CA,MU`，那么价格区间滑块和航司选择框就会自动被设置为对应的值。
- **状态的派发 (Dispatching/Updating)：**

- 当用户在筛选器中进行任何操作时（比如拖动价格滑块，或勾选一个新的航司），组件**不会**去更新一个内部的 `state`。
 - 相反，它会根据用户的操作，生成一个新的查询参数对象，然后调用 `setSearchParams(newParams)` 方法。这个方法会**更新URL**，但不会刷新整个页面。
 - `react-router` 检测到URL变化后，会触发相关组件的重新渲染。组件再次执行第一步“读取”逻辑，从新的URL中获取状态并更新UI。
 - 这就形成了一个清晰的单向数据流：**用户操作 → 更新URL → 组件重渲染 → 从URL读取新状态 → UI更新**。
 - **状态的持久化 (Persistence):**
 - 这个方案天然就解决了持久化问题。因为状态就在URL里，所以用户可以：
 - **刷新页面**：筛选条件不会丢失。
 - **复制链接分享给同事**：同事打开链接看到的是完全一样的筛选结果。
 - **收藏书签**：下次打开时可以恢复当时的状态。
 - **处理联动关系:**
 - 参数间的联动关系，被统一收敛在“派发”这个环节处理。
 - **例如**：当用户选择舱位为“商务舱”时，价格区间的最大值可能会自动放宽。这个逻辑被写在处理“舱位选择”的事件处理器中。当用户选择“商务舱”后，事件处理器会同时更新 `cabin=business` 和 `price=0,15000` 这两个参数，然后一起写入URL。这使得联动逻辑内聚，易于维护。
-

3、你在项目中是否处理过兼容性问题？可以举一个你印象最深刻的浏览器兼容性 case 吗？你是如何定位和解决的？

处理过非常多。印象最深刻的一个是在一个数据可视化看板页面遇到的 `ResizeObserver loop limit exceeded` 错误。

- **问题场景**：这个页面有一个复杂的栅格布局，用户可以拖拽调整每个卡片的大小。卡片内部有一个图表，图表的大小又会根据卡片的大小自适应。在某些特定布局下，当用户拖拽某个卡片的边缘时，页面会突然崩溃，并在控制台抛出 `ResizeObserver loop limit exceeded` 的错误。这个错误只在Chrome和Edge上出现，Firefox上则没有。
- **定位过程**：

- a. **初步分析：**这个错误直译过来是“ResizeObserver的循环超出了限制”。我立刻意识到，这很可能是因为在元素的尺寸变化，导致了另一个元素的尺寸变化，而后者又反过来影响了前者，形成了一个无限的“尺寸调整”循环。浏览器为了防止页面因此卡死，主动中断了这个循环并报错。
 - b. **使用Performance面板：**我在Chrome DevTools的Performance面板进行了录制，复现了这个操作。我看到在崩溃前的一瞬间，有非常密集的、连续不断的 `Layout`（布局）和 `Recalculate Style`（样式重计算）事件被触发，这印证了我的“无限循环”猜想。
 - c. **代码审查：**我检查了涉及到的卡片和图表组件的代码。我发现，我们的实现是：卡片组件通过 `ResizeObserver` 监听自身的尺寸变化，并将新尺寸通过 `props` 传递给内部的图表组件。而图表组件为了保持宽高比，在自身尺寸变化后，又会通过回调函数去尝试调整父级卡片的高度。这就是问题的根源。
- **解决方案：**
 - a. **打破循环：**解决方案是打破这个“父子相互影响尺寸”的循环。我们重新梳理了布局逻辑，规定**尺寸调整必须是单向的**。
 - b. **CSS优先：**我们最终改用CSS的 `aspect-ratio` 属性来控制图表的宽高比，而不是用JS去计算。这样，当父级卡片宽度变化时，图表的高度会由CSS自动计算，不再需要JS反向去通知父级，彻底切断了循环。
 - c. **对于不支持 `aspect-ratio` 的旧浏览器**，我们保留了JS计算的逻辑，但增加了 `debounce`（防抖）处理，确保尺寸调整的回调不会在高频拖拽中被连续触发，而是只在拖拽结束后执行一次，同样也避免了循环错误的发生。
-

4、追问几个细节。针对这个列表页，数据量非常大，你设计的数据获取策略是怎样的？是一次性请求所有数据，还是分页加载，或者是其他方式？各有什么优缺点？

对于机票搜索结果这种动辄成百上千条的数据，设计合理的数据获取策略至关重要。

- **一次性请求所有数据：**
 - **优点：**实现最简单。
 - **缺点：绝对不可行。**会导致首个请求的响应时间极长（可能几十秒甚至超时），传输的数据量巨大（可能几MB），并且在前端一次性渲染上千个DOM节点会造成浏览器长时间卡死。用户体验是灾难性的。
- **分页加载 (Pagination)：**

- **优点：**业界标准，实现简单。每次只请求一页的数据（如20条），请求响应快，前端渲染压力小。用户可以明确知道自己在哪一页，也可以直接跳转到指定页码。
- **缺点：**对于“探索式”的浏览场景，用户体验并不流畅。每次点击“下一页”都需要等待加载，页面会刷新，滚动位置会丢失。
- **无限滚动 (Infinite Scroll)：**
 - **优点：**用户体验流畅、现代。用户只需向下滚动，新数据就会自动加载并追加到列表末尾，感觉列表是“无限”的，非常适合移动端和探索式浏览。
 - **缺点：**用户无法感知列表的总体长度，也无法直接跳转到中间或末尾。如果实现不当（没有结合虚拟列表），会导致DOM节点无限增多，最终拖垮页面。

我的设计策略：分页API + 前端无限滚动体验

我们的最终选择是结合两者的优点：**后端提供分页接口，前端实现无限滚动。**

1. **后端接口：**后端依然提供标准的分页API，例如 `/api/flights?page=1&pageSize=20`。这对于后端来说是最简单、最高效的。
2. **前端实现：**
 - 前端首次加载时，请求第一页数据（`page=1`）。
 - 前端维护一个 `page` 的计数器状态。
 - 当用户滚动到页面底部时，触发加载下一页的逻辑，`page` 计数器加一，然后请求 `/api/flights?page=2`。
 - 请求成功后，**将新获取的数据追加（append）**到现有列表的末尾，而不是替换。
 - 这样，对用户来说，体验是无缝的无限滚动；对后端来说，依然是清晰、无状态的分页查询。

5、用户在左侧的筛选器中（比如选择航司、起飞时段、价格区间），列表需要实时更新。这个筛选功能的逻辑你打算放在前端还是后端实现？为什么？如果放在前端，如何保证筛选的性能？

对于机票搜索这种场景，这个筛选逻辑**必须放在后端实现。**

为什么必须是后端：

1. **数据全集问题：**筛选的前提是拥有**所有**数据的全集。一个航线搜索的结果可能有成千上万条，让前端一次性把这些数据全部下载下来（哪怕只是ID和价格），再在浏览器里进行筛选，这是完全不现实的，会耗尽用户的内存和带宽。
2. **数据的一致性与实时性：**机票的价格和余票信息是瞬息万变的。只有后端才能连接到最新、最准确的数据源（GDS或航司直连）。在前端做筛选，操作的是一个已经过时的数据快照。

3. **性能与效率**：数据库系统（如Elasticsearch, PostgreSQL）为数据筛选、排序和聚合做了极致的优化。同样是筛选出“价格低于5000元，东方航空执飞”的航线，数据库在百万条数据中完成可能只需要几十毫秒，而让前端JS去处理，性能会差几个数量级。
4. **业务逻辑的复杂性**：筛选逻辑可能包含复杂的业务规则（如“计算中转时间”、“判断共享航班”），这些逻辑理应属于后端业务范畴。

前端的角色：

前端的角色不是“执行筛选”，而是**“构建筛选参数”**。前端UI负责让用户方便地选择筛选条件，然后将这些条件组合成一个结构化的请求参数，发送给后端API，例如：`GET /api/flights?airlines=CA,MU&priceRange=0,5000&departureTime=morning`。后端根据这些参数，返回筛选、排序后的那一页数据。

如果（在某些数据量极小的情况下）放在前端，如何保证性能？

虽然不适用于本场景，但如果数据量真的很少（比如几百条），并且可以一次性加载，那么前端筛选的性能优化可以这样做：

1. **避免在渲染函数中直接筛选**：不要在组件的render方法里直接调用 `.filter()`。这会导致每次组件因任何原因重渲染时，都重新计算一遍。
2. **使用 `useMemo`**：将筛选逻辑和结果用 `useMemo` 包裹起来。只有当原始列表或筛选条件发生变化时，才重新执行筛选计算，缓存计算结果。
3. **优化数据结构**：如果筛选频繁，可以将原始的数组数据，转换成更便于查找的数据结构，比如按某个关键字段建立索引（Map）。
4. **使用Web Worker**：如果筛选逻辑本身极其复杂（比如涉及地理位置计算等），可以将计算任务放到Web Worker中，防止阻塞UI线程。

6、当用户向下滚动页面时，如何实现“无限滚动”加载更多数据？在实现这个功能时，有哪些需要注意的性能和体验上的细节？比如如何防止重复发送请求？

实现无限滚动的最佳现代实践是使用 `Intersection Observer` API。

实现思路：

1. 在列表的末尾，放置一个不可见的“哨兵”元素（Sentinel Element），比如一个高度为1px的 `<div>`。
2. 使用 `Intersection Observer` 来观察这个“哨兵”元素。
3. 当这个“哨兵”元素进入或即将进入用户的视口时，`Intersection Observer` 的回调函数就会被触发。

4. 在这个回调函数中，执行加载下一页数据的逻辑。
5. 这种方式比监听 `scroll` 事件性能好得多，因为它不在高频的滚动中反复计算，而是由浏览器在“哨兵”元素可见性变化时才通知我们。

需要注意的性能和体验细节：

1. **防止重复发送请求：**这是最重要的一点。我们需要一个状态来跟踪加载状态。
 - 在组件中维护一个 `isLoading` 的状态，默认为 `false`。
 - 在触发加载逻辑时，首先检查 `if (isLoading) return;`，如果正在加载中，则直接返回，不执行任何操作。
 - 在发起API请求前，立刻将 `isLoading` 设为 `true`。
 - 在API请求完成（无论是成功还是失败）后，在 `finally` 块中，将 `isLoading` 设回 `false`。这确保了即使请求失败，用户依然可以再次触发加载。
 2. **处理加载状态的UI：**
 - 当 `isLoading` 为 `true` 时，应该在列表的末尾显示一个加载指示器（Loading Spinner），给用户明确的反馈。
 3. **处理数据加载到底的情况：**
 - API应该返回信息告诉我们是否还有更多数据。或者，如果某次请求返回的数据为空或少于每页数量，我们就知道已经到底了。
 - 需要一个 `hasMore` 的状态。当加载到底后，将其设为 `false`。
 - 当 `hasMore` 为 `false` 时，不再触发加载逻辑，并将“哨兵”元素移除或禁用观察，同时在列表末尾显示“没有更多数据了”的提示。
 4. **滚动位置的保持：**
 - 当新数据加载并追加到列表中时，要确保用户的滚动位置不会“跳动”。通常，只要是向下追加内容，浏览器能很好地处理。但需要注意避免因新内容渲染导致上方内容布局变化。
 5. **与虚拟列表结合：**
 - 对于真正海量的无限滚动，必须结合虚拟列表。否则，随着用户不断滚动，DOM节点会无限增多，最终导致页面崩溃。无限滚动负责“获取数据”，虚拟列表负责“高效渲染”。
-

7、最后一个问题，对于这个页面的可维护性，你会如何设计你的组件结构？比如单个航线卡片、筛选器、排序栏等，你会如何把它们抽象成可复用、低耦合的组件？

我会遵循**容器组件（Container Components）与展示组件（Presentational Components）**的设计模式，并强调组件的单一职责原则。

组件结构设计如下：

1. FlightSearchPage (容器组件 / 页面级)

- 这是“最聪明”的顶层组件。
- **职责：**它不关心UI长什么样。它的唯一职责是**管理状态和逻辑**。
 - 从URL读取筛选和排序参数。
 - 管理 `isLoading` , `error` , `hasMore` 等API交互状态。
 - 发起API请求获取数据。
 - 定义 `handleFilterChange` , `handleSortChange` , `loadMore` 等回调函数。
- 它会将状态数据和回调函数，通过 `props` 传递给下面的展示组件。

2. FilterPanel (展示组件 / 可复用)

- 这是一个“很笨”但可复用的组件。
- **职责：**只负责**渲染筛选器UI**。
- 它接收 `currentFilters` （当前筛选值）和 `onFilterChange` （当筛选变化时调用的函数）作为 `props` 。
- 它内部不维护任何复杂的业务逻辑状态。当用户点击一个复选框时，它只是调用 `props.onFilterChange(...)` ，把新的筛选值通知给父组件(`FlightSearchPage`)。

3. SortBar (展示组件 / 可复用)

- 与 `FilterPanel` 类似。接收 `currentSort` 和 `onSortChange` 作为 `props` 。只负责渲染排序选项并向上传递事件。

4. FlightList (展示组件 / 业务相关)

- **职责：**负责**渲染整个航班列表**。
- 它接收 `flights` （航班数据数组）、`isLoading` , `hasMore` 等 `props` 。
- 它内部会包含“无限滚动”的逻辑，并在需要时调用从 `props` 接收的 `loadMore` 函数。
- 它会遍历 `flights` 数组，为每一项渲染一个 `FlightCard` 组件。

5. FlightCard (展示组件 / 原子级可复用)

- 这是最纯粹的展示组件。
- **职责：**只负责**渲染单条航线信息**。
- 它接收一个 `flight` 对象作为 `prop` ，然后把这个对象里的数据显示在正确的位置上。它完全不知道这个数据是怎么来的，也不知道点击它会发生什么，所有交互都通过回调函数 `props` 向外通知。

这样设计的好处：

- **高内聚，低耦合：**状态逻辑集中在 `FlightSearchPage`，UI渲染分散在各个展示组件。职责清晰，互不干扰。
- **强可复用性：**`FilterPanel`，`SortBar`，`FlightCard` 这些组件不与任何特定的数据获取逻辑绑定，可以轻松地被用在“酒店搜索页”或“火车票搜索页”上，只需要传入符合其接口的 `props` 即可。
- **易于测试：**展示组件因为是纯粹的输入输出（`props` 进，UI出），所以非常容易进行单元测试或 Storybook快照测试。容器组件的逻辑也可以被独立测试。
- **易于维护和协作：**不同的开发者可以并行地开发不同的组件。当需要修改筛选逻辑时，只需要关心 `FlightSearchPage` 和 `FilterPanel`，而不会影响到列表的渲染。